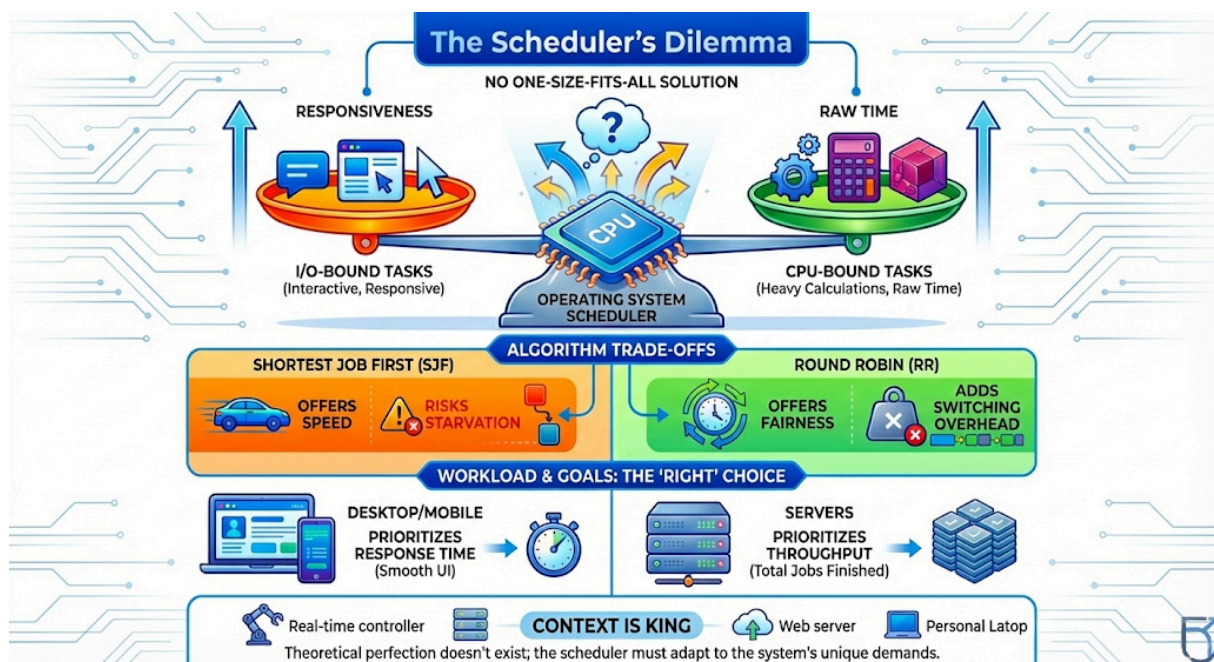


# Lecture 09 NOTES– CPU Scheduling Evaluation and Advanced Topics

## Introduction: The Scheduler's Dilemma

The Operating System scheduler faces a constant balancing act. There is no "one-size-fits-all" solution to managing the CPU.

- **Diverse Processes:** The OS must juggle **I/O-bound** tasks (interactive apps needing responsiveness) and **CPU-bound** tasks (heavy calculations needing raw time). They have conflicting needs.



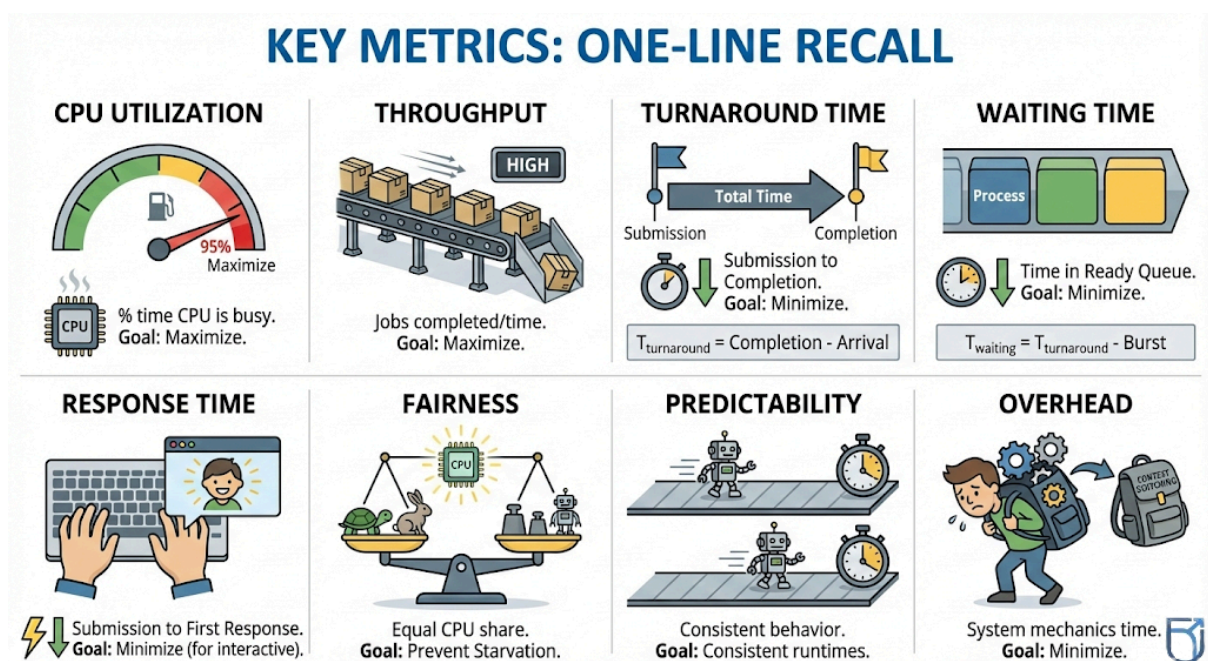
- **No Single "Best" Algorithm:** Every algorithm involves a trade-off.
  - *Shortest Job First* offers speed but risks starvation.
  - *Round Robin* offers fairness but adds switching overhead.
- **Workload & Goals:** The "right" choice depends on the objective.
  - **Desktop/Mobile:** Prioritizes **Response Time** (smooth UI).
  - **Servers:** Prioritizes **Throughput** (total jobs finished).
- **Context is King:** Theoretical perfection doesn't exist; the scheduler must adapt to whether the system is a real-time controller, a web server, or a personal laptop.

## Evaluation Framework: How to Compare Schedulers

When choosing a CPU scheduling algorithm, we use these standard metrics to measure performance.

## Key Metrics (One-Line Recall)

- **CPU Utilization:**
  - *Meaning:* % time the CPU is busy.
  - *Goal:* Maximize (keep the CPU working as much as possible).
- **Throughput:**
  - *Meaning:* Number of jobs completed per unit of time.
  - *Goal:* Maximize (finish more work in less time).
- **Turnaround Time:**
  - *Meaning:* Total time taken from submission to completion.
  - *Formula:* Completion Time - Arrival Time
  - *Goal:* Minimize.
- **Waiting Time:**
  - *Meaning:* Total time a process spends waiting in the Ready Queue.
  - *Formula:* Turnaround Time - Burst Time
  - *Goal:* Minimize.



- **Response Time:**
  - *Meaning:* Time from submission until the *first* response is produced.
  - *Crucial for:* Interactive systems (gaming, typing).
  - *Goal:* Minimize.
- **Fairness:**
  - *Meaning:* Ensuring every process gets a fair share of the CPU.
  - *Goal:* Prevent starvation.
- **Predictability:**
  - *Meaning:* Consistent behavior for similar jobs.
  - *Goal:* A job should take roughly the same time to run regardless of system load.
- **Overhead:**

- *Meaning:* Time wasted on system mechanics (like context switching) instead of actual work.
- *Goal:* Minimize.

## Algorithm Comparison Insight

Here is a quick breakdown of how the major scheduling algorithms stack up against each other based on their strengths and weaknesses.

| Algorithm                                 | Key Characteristic       | Pros                                                                          | Cons                                                                                       |
|-------------------------------------------|--------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>FCFS</b><br>(First-Come, First-Served) | <b>Simple Queue</b>      | Easy to implement; fair in arrival order.                                     | <b>Inefficient.</b> Causes the "Convoy Effect" (short jobs wait behind long ones).         |
| <b>SJF</b> (Shortest Job First)           | <b>Burst Time</b>        | <b>Best Waiting Time.</b> Mathematically optimal for minimizing average wait. | <b>Poor Response Time.</b> Hard to predict burst time; long jobs can starve.               |
| <b>RR</b> (Round Robin)                   | <b>Time Quantum</b>      | <b>Best Response Time.</b> Fair; no starvation; ideal for time-sharing.       | <b>Moderate Overhead.</b> Context switching costs can be high if the quantum is too small. |
| <b>Priority</b>                           | <b>Rank / Importance</b> | <b>Efficient.</b> Handles urgent tasks (like system alerts) immediately.      | <b>Unfair.</b> Low-priority tasks suffer from starvation (solved by "Aging").              |

## Numerical Evaluation and Gantt Examples

**Q1:** Consider the following set of processes with their Arrival Times (AT), Burst Times (BT), and Priorities (lower number = higher priority).

| Process | Arrival Time (AT) | Burst Time (BT) | Priority |
|---------|-------------------|-----------------|----------|
| P1      | 0                 | 10              | 3        |
| P2      | 1                 | 1               | 1        |
| P3      | 2                 | 2               | 2        |
| P4      | 3                 | 1               | 4        |
| P5      | 4                 | 5               | 2        |

**Solution:-**

### **FCFS (First-Come, First-Served)**

#### **Core Logic**

- **Principle:** The "Queue" Logic. The CPU is assigned strictly in the order that processes arrive.
- **Non-Preemptive:** Once a process starts, it keeps the CPU until it is fully finished. The OS cannot interrupt it.
- **The Flaw (Convoy Effect):** If the first process is very long (like P1 here), all subsequent short processes (P2, P3, P4) get stuck waiting, leading to high average waiting times.

#### **Execution Sequence**

Since the queue follows FIFO (First-In, First-Out), the order is simply the Arrival Time order.

- **Time 0:** P1 arrives and starts immediately. (Runs for 10ms).
- **Time 10:** P1 finishes. P2 is next in line. (Runs for 1ms).
- **Time 11:** P2 finishes. P3 is next. (Runs for 2ms).
- **Time 13:** P3 finishes. P4 is next. (Runs for 1ms).
- **Time 14:** P4 finishes. P5 is next. (Runs for 5ms).
- **Time 19:** P5 finishes.

**Gantt Chart:**

**This table represents the timeline of the CPU usage.**



| Time Interval | Process ID |
|---------------|------------|
| 0 – 10        | P1         |
| 10 – 11       | P2         |
| 11 – 13       | P3         |
| 13 – 14       | P4         |
| 14 – 19       | P5         |

## Calculation Table

We use the standard formulas:

- Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)
- Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)

| Process | Arrival Time (AT) | Burst Time (BT) | Completion Time (CT) | Turnaround Time (TAT) | Waiting Time (WT) |
|---------|-------------------|-----------------|----------------------|-----------------------|-------------------|
| P1      | 0                 | 10              | 10                   | 10                    | 0                 |
| P2      | 1                 | 1               | 11                   | 10                    | 9                 |
| P3      | 2                 | 2               | 13                   | 11                    | 9                 |
| P4      | 3                 | 1               | 14                   | 11                    | 10                |

|    |   |   |    |    |    |
|----|---|---|----|----|----|
| P5 | 4 | 5 | 19 | 15 | 10 |
|----|---|---|----|----|----|

## Final Averages

To evaluate the performance of this algorithm:

### Average Waiting Time (AWT):

$$\text{AWT} = (0 + 9 + 9 + 10 + 10) / 5 = 38 / 5 = 7.6 \text{ ms}$$

### Average Turnaround Time (ATAT):

$$\text{ATAT} = (10 + 10 + 11 + 11 + 15) / 5 = 11.4 \text{ ms}$$

**Key Observation:** Notice that P2 only needed **1ms** to run, but it had to wait **9ms** just because P1 arrived slightly earlier. This is the classic **Convoy Effect**.

## SJF (Shortest Job First – Non-Preemptive)

### Core Logic

- **Principle:** The "Efficiency" Logic. When the CPU is free, it looks at the pool of available processes and picks the one with the **shortest Burst Time**.
- **Tie-Breaking Rule:** If two processes have the same burst time, the one that **arrived first** gets priority.
- **Non-Preemptive:** Once selected, the process runs to completion.

### Execution Sequence

- **Time 0:** Only **P1** is available. It starts (Runs 10ms).
- **Time 10:** P1 finishes. Available in Queue: P2 (1ms), P3 (2ms), P4 (1ms), P5 (5ms).
  - *Decision:* P2 and P4 both need 1ms. Since P2 arrived at T=1 and P4 at T=3, **P2** wins the tie.
- **Time 11:** P2 finishes. Remaining: P3 (2ms), P4 (1ms), P5 (5ms).
  - *Decision:* **P4** is the shortest (1ms).
- **Time 12:** P4 finishes. Remaining: P3 (2ms), P5 (5ms).
  - *Decision:* **P3** is the shortest (2ms).
- **Time 14:** P3 finishes. Only **P5** is left (5ms).
- **Time 19:** P5 finishes.

**Gantt Chart:**

| Time Interval | Process ID |
|---------------|------------|
| 0 – 10        | P1         |
| 10 – 11       | P2         |
| 11 – 12       | P4         |
| 12 – 14       | P3         |
| 14 – 19       | P5         |

- **Calculation Table**

Formula:  $WT = TAT - BT$

| Process | Arrival Time (AT) | Burst Time (BT) | Completion Time (CT) | Turnaround Time (TAT) | Waiting Time (WT) |
|---------|-------------------|-----------------|----------------------|-----------------------|-------------------|
| P1      | 0                 | 10              | 10                   | 10                    | 0                 |
| P2      | 1                 | 1               | 11                   | 10                    | 9                 |
| P3      | 2                 | 2               | 14                   | 12                    | 10                |
| P4      | 3                 | 1               | 12                   | 9                     | 8                 |

|    |   |   |    |    |    |
|----|---|---|----|----|----|
| P5 | 4 | 5 | 19 | 15 | 10 |
|----|---|---|----|----|----|

## Final Averages

- Average Waiting Time:  $(0 + 9 + 10 + 8 + 10) / 5 = 7.4\text{ms}$

## Round Robin (Time Quantum = 3)

### Core Logic

- **Principle:** The "Fairness" Logic. Every process gets a strict time limit (Quantum = 3ms).
- **Rule:** If a process isn't finished within 3ms, it is paused and moved to the back of the queue. If it finishes early (like P2, P3, P4), the CPU immediately moves to the next process.

### Execution Sequence

- **0-3: P1** runs (3ms). Remaining: 7. Moves to back.
- **3-4: P2** runs. It only needs 1ms. **Finishes.**
- **4-6: P3** runs. It only needs 2ms. **Finishes.**
- **6-9: P1** gets the CPU again (3ms). Remaining: 4. Moves to back.
- **9-10: P4** runs. It only needs 1ms. **Finishes.**
- **10-13: P5** runs (3ms). Remaining: 2. Moves to back.
- **13-16: P1** runs (3ms). Remaining: 1. Moves to back.
- **16-18: P5** runs (remaining 2ms). **Finishes.**
- **18-19: P1** runs (remaining 1ms). **Finishes.**

### Gantt Chart:

| Time Interval | Process ID | Notes              |
|---------------|------------|--------------------|
| 0 – 3         | P1         | Time slice expired |
| 3 – 4         | P2         | Finished early     |

|         |    |                    |
|---------|----|--------------------|
| 4 – 6   | P3 | Finished early     |
| 6 – 9   | P1 | Time slice expired |
| 9 – 10  | P4 | Finished early     |
| 10 – 13 | P5 | Time slice expired |
| 13 – 16 | P1 | Time slice expired |
| 16 – 18 | P5 | Finished           |
| 18 – 19 | P1 | Finished           |

#### Waiting Time Formula:

Waiting Time = Finish Time – Arrival Time – Burst Time

| Process | AT | BT | Completion Time (CT) | Turnaround Time (TAT) | Waiting Time (WT) |
|---------|----|----|----------------------|-----------------------|-------------------|
| P1      | 0  | 10 | 19                   | 19                    | 9                 |
| P2      | 1  | 1  | 4                    | 3                     | 2                 |
| P3      | 2  | 2  | 6                    | 4                     | 2                 |



|           |   |   |    |    |   |
|-----------|---|---|----|----|---|
| <b>P4</b> | 3 | 1 | 10 | 7  | 6 |
| <b>P5</b> | 4 | 5 | 18 | 14 | 9 |

## Final Averages

- **Average Waiting Time:**  $(9 + 2 + 2 + 6 + 9) / 5 = 5.6$  ms
- **Observation:** This algorithm has the highest number of Context Switches (8 switches), which creates overhead, but it provides the best response time for users.

## Preemptive Priority Scheduling

(Lower priority number means higher priority)

## Core Logic

- Principle: The "Urgency" Logic. The CPU is always owned by the process with the highest priority (Lowest Number).
- Preemption: If a "VIP" (higher priority process) arrives, the current process is kicked off immediately.

## Execution Sequence

- **0-1: P1** (Prio 3) runs. At T=1, **P2** (Prio 1) arrives.  $P2 < P1$ . **Preempt.**
- **1-2: P2** runs. Finishes. At T=2, **P3** (Prio 2) arrives.  $P3 < P1$ . **P3 runs.**
- **2-4: P3** runs. Finishes. At T=4, **P5** (Prio 2) arrives.  $P5 < P1$ . **P5 runs.**
- **4-9: P5** runs. Finishes.
- **9-18:** Only P1 (Prio 3) and P4 (Prio 4) are left. P1 is higher priority. **P1 resumes.**
- **18-19: P4** runs. Finishes.

## Gantt Chart:

| Time Interval | Process ID | Reason for Switch |
|---------------|------------|-------------------|
|               |            |                   |

|                |           |                 |
|----------------|-----------|-----------------|
| <b>0 – 1</b>   | <b>P1</b> | Preempted by P2 |
| <b>1 – 2</b>   | <b>P2</b> | Finished        |
| <b>2 – 4</b>   | <b>P3</b> | Finished        |
| <b>4 – 9</b>   | <b>P5</b> | Finished        |
| <b>9 – 18</b>  | <b>P1</b> | Finished        |
| <b>18 – 19</b> | <b>P4</b> | Finished        |

**Waiting Time Calculation:**

| <b>Process</b> | <b>AT</b> | <b>BT</b> | <b>Completion Time (CT)</b> | <b>Turnaround Time (TAT)</b> | <b>Waiting Time (WT)</b> |
|----------------|-----------|-----------|-----------------------------|------------------------------|--------------------------|
| <b>P1</b>      | 0         | 10        | 18                          | 18                           | 8                        |
| <b>P2</b>      | 1         | 1         | 2                           | 1                            | 0                        |
| <b>P3</b>      | 2         | 2         | 4                           | 2                            | 0                        |
| <b>P4</b>      | 3         | 1         | 19                          | 16                           | 15                       |
| <b>P5</b>      | 4         | 5         | 9                           | 5                            | 0                        |

## Final Averages

- **Average Waiting Time:**  $(8 + 0 + 0 + 15 + 0) / 5 = 4.6\text{ms}$

## Comparative Analysis & Conclusion

Based on the results above, fill in the comparison table:

| Algorithm   | Avg Waiting  | Context Switches | Key Characteristic                                              |
|-------------|--------------|------------------|-----------------------------------------------------------------|
| FCFS        | 7.6          | 0                | Simple but suffers from the <b>Convoy Effect</b> .              |
| SJF         | 7.4          | 0                | Good throughput, but potential <b>Starvation</b> for long jobs. |
| Round Robin | 5.6          | 8 (High)         | Best <b>Response Time</b> ; Ideal for Time-Sharing Systems.     |
| Priority    | 4.8 (Lowest) | 6                | Lowest waiting time, but <b>P4 Starved</b> (waited 15s).        |

### Final Verdict:

- Use **Round Robin** for Desktop/Mobile (UI needs to be snappy).
- Use **SJF/Priority** for Batch Systems (where throughput matters more than interactivity).

## Workload Characteristics

The "best" algorithm depends entirely on the type of work the CPU is doing.

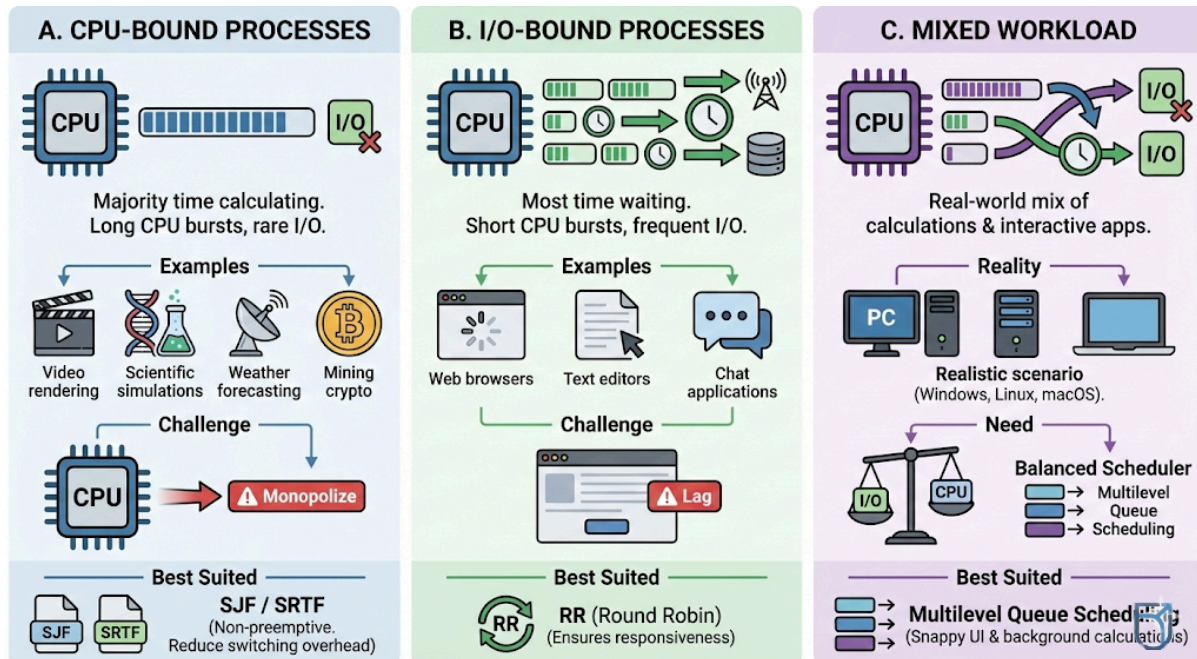
### A. CPU-Bound Processes

These processes spend the vast majority of their time performing calculations.

- **Characteristics:** Long CPU bursts; very rare I/O operations.
- **Examples:** Video rendering, scientific simulations, weather forecasting, mining crypto.

- **Challenge:** They can monopolize the CPU if not managed.

**Best Suited: SJF / SRTF** (Non-preemptive algorithms work well here to reduce switching overhead, as response time is less critical).



## B. I/O-Bound Processes

These processes spend most of their time waiting for input or output.

- **Characteristics:** Very short CPU bursts; frequent I/O waits (disk, network, user input).
- **Examples:** Web browsers, text editors, chat applications.
- **Challenge:** They need the CPU *immediately* when ready, or the UI lags.

**Best Suited: RR (Round Robin)** (Ensures frequent checks so the UI remains responsive).

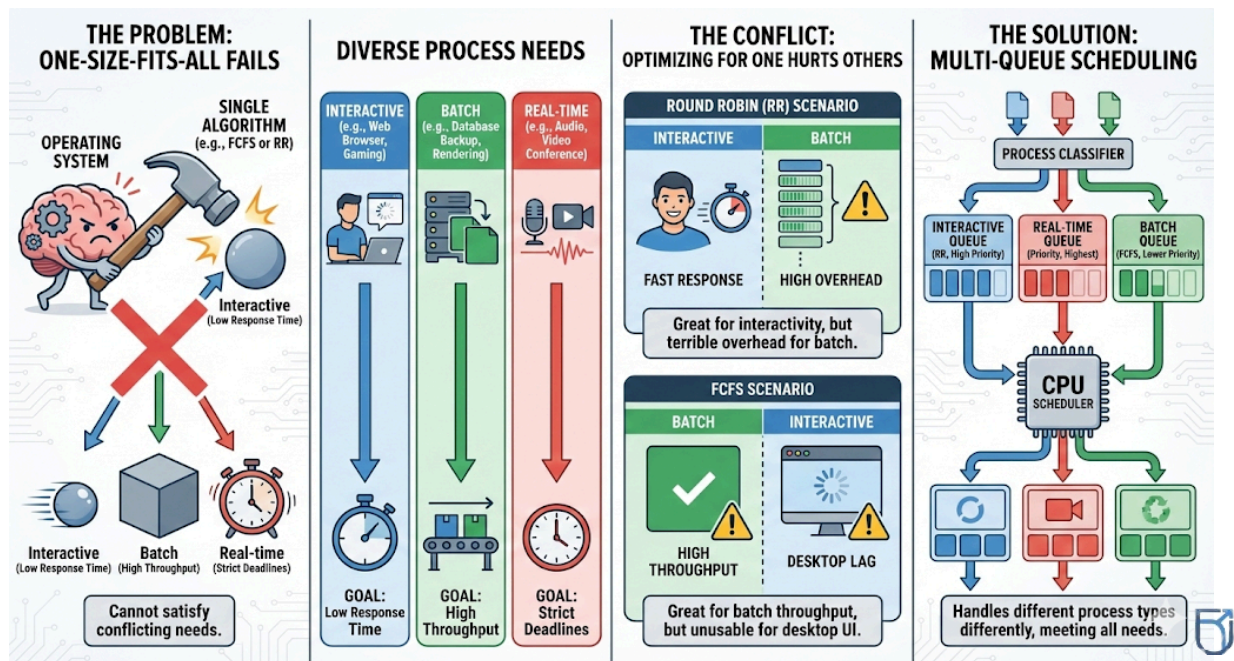
## C. Mixed Workload

A real-world system usually contains a chaotic mix of both types.

- **Characteristics:** A combination of long-running calculations and interactive apps.
- **Reality:** This is the most realistic scenario for general-purpose OS (Windows, Linux, macOS).
- **Need:** A balanced scheduler (like Multilevel Queue Scheduling) that can distinguish between the two, giving priority to I/O-bound tasks to keep the system snappy while letting CPU-bound tasks run in the background.

## Why One Algorithm Is Not Enough

A single scheduling algorithm (like just FCFS or just Round Robin) cannot satisfy the conflicting needs of a modern operating system.



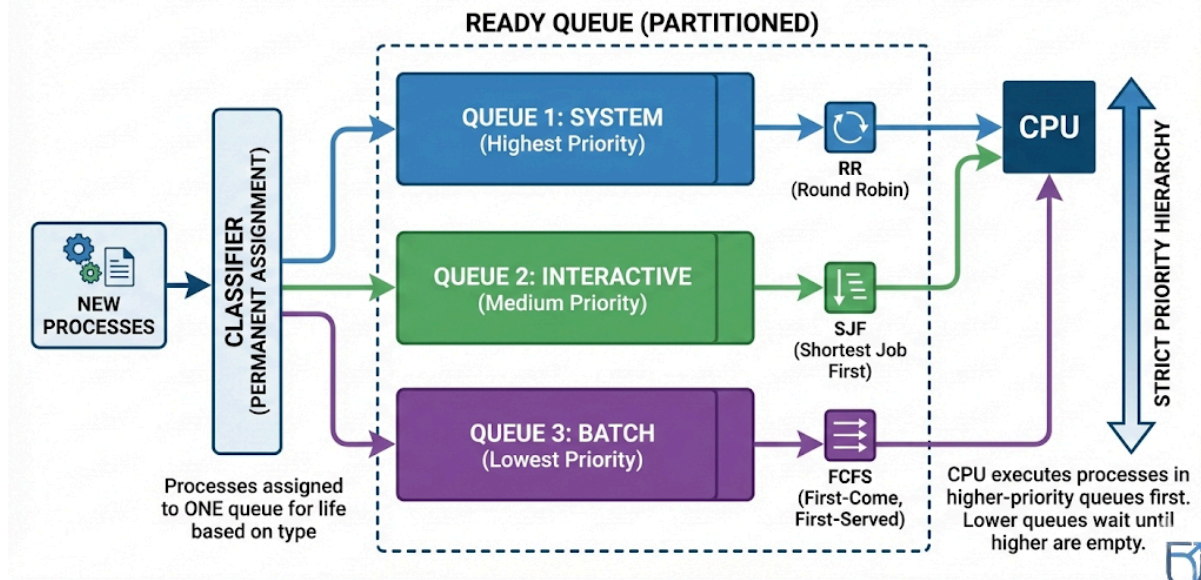
- **Different Needs:** Interactive processes (like a web browser) need low **Response Time**. Batch processes (like a nightly database backup) care about **Throughput** and don't mind waiting. Real-time processes (like audio processing) need strict deadlines.
- **The Conflict:** Optimizing for one usually hurts the other. Round Robin is great for interactivity but terrible for batch processing overhead. FCFS is great for batch throughput but unusable for a desktop UI.
- **The Solution:** This necessitates a system that can handle different types of processes differently—leading us to **Multi-Queue Scheduling**.

## Fixed Multi-Level Queue Scheduling (MLQ)

### Concept

In this approach, the Ready Queue is no longer a single line. Instead, it is partitioned into several separate queues.

## FIXED MULTI-LEVEL QUEUE SCHEDULING (MLQ) CONCEPT



- **Classification:** Processes are permanently assigned to one specific queue based on their type (e.g., System, Interactive, Background) immediately upon arrival.
- **Independent Algorithms:** Each queue can use its own scheduling algorithm.
  - *Foreground (Interactive) Queue* might use **Round Robin (RR)** for responsiveness.
  - *Background (Batch) Queue* might use **FCFS** for efficiency.
- **Fixed Priority:** There is a strict hierarchy between the queues. The CPU will only run a process from the Background queue if the Foreground queue is completely empty.

### Example Queues

1. **System Processes** (Highest Priority) - Vital OS tasks.
2. **Interactive Processes** - User applications (Word, Chrome).
3. **Batch Processes** (Lowest Priority) - Background compilations, backups.

### Problems with MLQ

- **Rigidity:** A process is assigned to a queue for life. If a compiler (Batch) suddenly becomes interactive (an error pops up), it is stuck in the slow queue.
- **Starvation:** If higher-priority interactive processes keep arriving, the batch processes at the bottom may never execute.

## The Big Classification Problem

MLQ relies on the OS knowing what *type* of process something is right when it starts. This is a major flaw.

- **Unpredictability:** You cannot know a process's behavior just by looking at its name. A "video editor" might be interactive one moment (UI editing) and CPU-bound the next (rendering).

- **Dynamic Behavior:** Processes change over time. A process might start as CPU-intensive (loading data) and then become I/O-intensive (waiting for user input).
- **The Failure:** Because MLQ assigns queues statically, it cannot adapt to these changes.

## Multi-Level Feedback Queue (MLFQ)

This is the evolution of MLQ that solves the rigidity problem. It is the algorithm used by most modern general-purpose operating systems (like Unix, Windows, and macOS).

### Core Idea

Instead of forcing a process into a queue based on a guess, we **let the process's behavior decide its priority.**

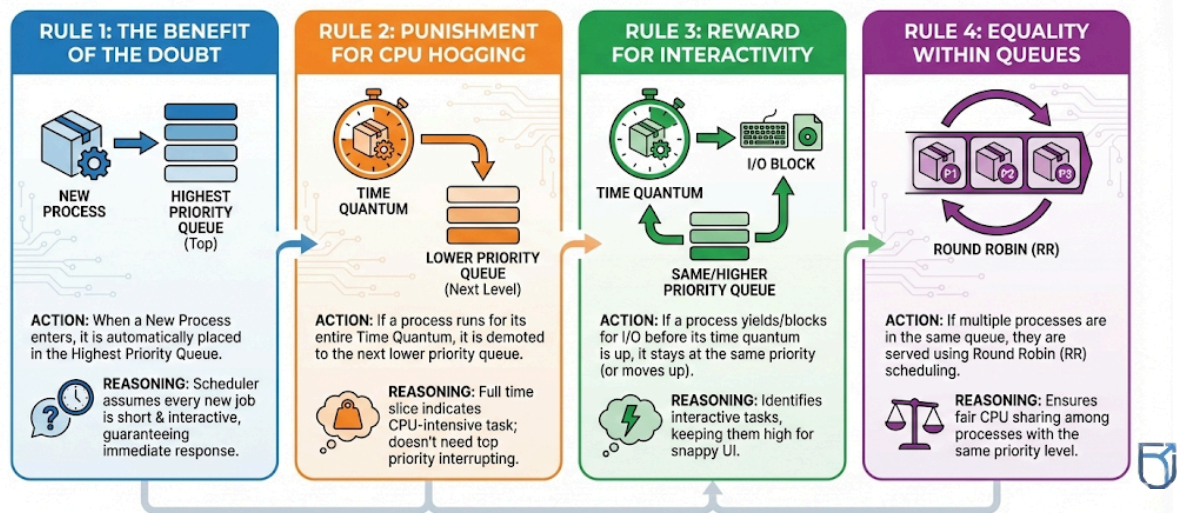
- **Dynamic Movement:** Processes are not stuck in one queue. They can move up or down between queues.
- **No Prediction Needed:** The scheduler does not need to know if a job is "short" or "long" beforehand. It "learns" by watching the process run.
- **The Rule:**
  - If a process uses too much CPU time (CPU-bound), it is moved to a lower-priority queue.
  - If a process waits for I/O (Interactive), it stays in or moves to a high-priority queue.

## MLFQ Rules (Core Rules)

1. **Rule 1: The Benefit of the Doubt**
  - **Action:** When a **New Process** enters the system, it is automatically placed in the **Highest Priority Queue**.
  - *Reasoning:* The scheduler assumes every new job is a short, interactive task until proven otherwise. This guarantees immediate response time for new apps.
2. **Rule 2: Punishment for CPU Hogging**
  - **Action:** If a process runs for its **entire Time Quantum** without pausing, it is **demoted** to the next lower priority queue.
  - *Reasoning:* Using the full time slice indicates the process is CPU-intensive (like a video render). It doesn't need to be at the top interrupting the user.



## MLFQ RULES (CORE RULES)



### 3. Rule 3: Reward for Interactivity

- **Action:** If a process gives up the CPU voluntarily (yields or blocks for I/O) **before** its time quantum is up, it **stays at the same priority level** (or in some variations, moves up).
- **Reasoning:** This behavior identifies interactive tasks (like a keyboard waiting for input). We want to keep these high up so the UI stays snappy.

### 4. Rule 4: Equality Within Queues

- **Action:** If multiple processes are in the **Same Priority Queue**, they are scheduled using **Round Robin (RR)**.

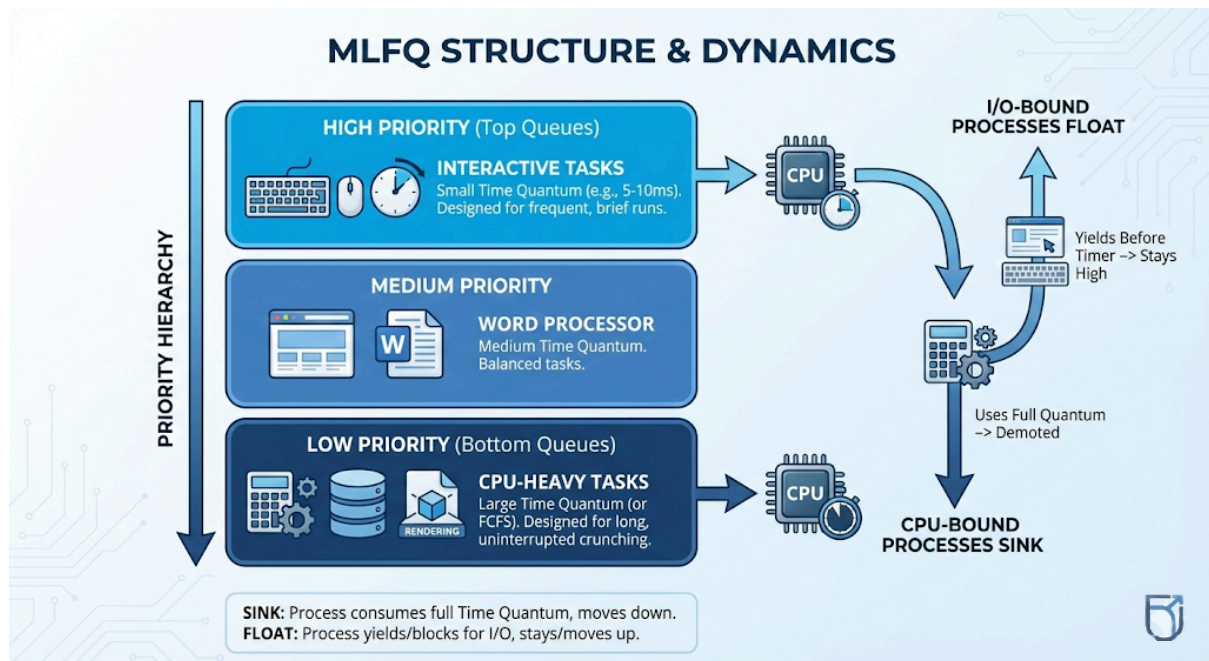
## MLFQ Structure

The queues in an MLFQ are not identical. They are designed to handle the specific types of processes that end up in them.

### • The Hierarchy:

- **Top Queues (High Priority):** These have a very **Small Time Quantum** (e.g., 5-10ms). They are designed for interactive tasks that need to run frequently but briefly.
- **Bottom Queues (Low Priority):** These have a very **Large Time Quantum** (or sometimes use FCFS). They are designed for CPU-heavy tasks that need long, uninterrupted stretches to crunch numbers.





- **The Dynamics (Sink vs. Float):**

- **CPU-Bound Processes Sink:** A long calculation will repeatedly use its full quantum and be demoted step-by-step until it hits the bottom, where it can run efficiently in the background.
- **I/O-Bound Processes Float:** Interactive apps usually finish quickly (before the timer rings) and thus "float" or stay at the top, ensuring they remain responsive to the user.

## MLFQ Advantages

- **Adaptive Scheduling:** It doesn't need to know if a process is "short" or "long" beforehand. It learns and adapts automatically.
- **Good Response Time:** By keeping I/O-bound tasks at the top, it ensures the system feels fast and responsive (like Round Robin).
- **High Throughput:** By moving CPU-bound tasks to the bottom with longer time slices, it reduces context switching overhead for heavy jobs (like FCFS/SJF).
- **Prevents Starvation (with Aging):** *Note:* Basic MLFQ can suffer from starvation if too many interactive jobs arrive. To fix this, most real-world MLFQ implementations add a "Periodic Boost" (Aging) rule: *After a certain time period, move ALL processes back to the top queue.* This resets the system and guarantees low-priority jobs eventually run.

## Gaming the Scheduler Problem

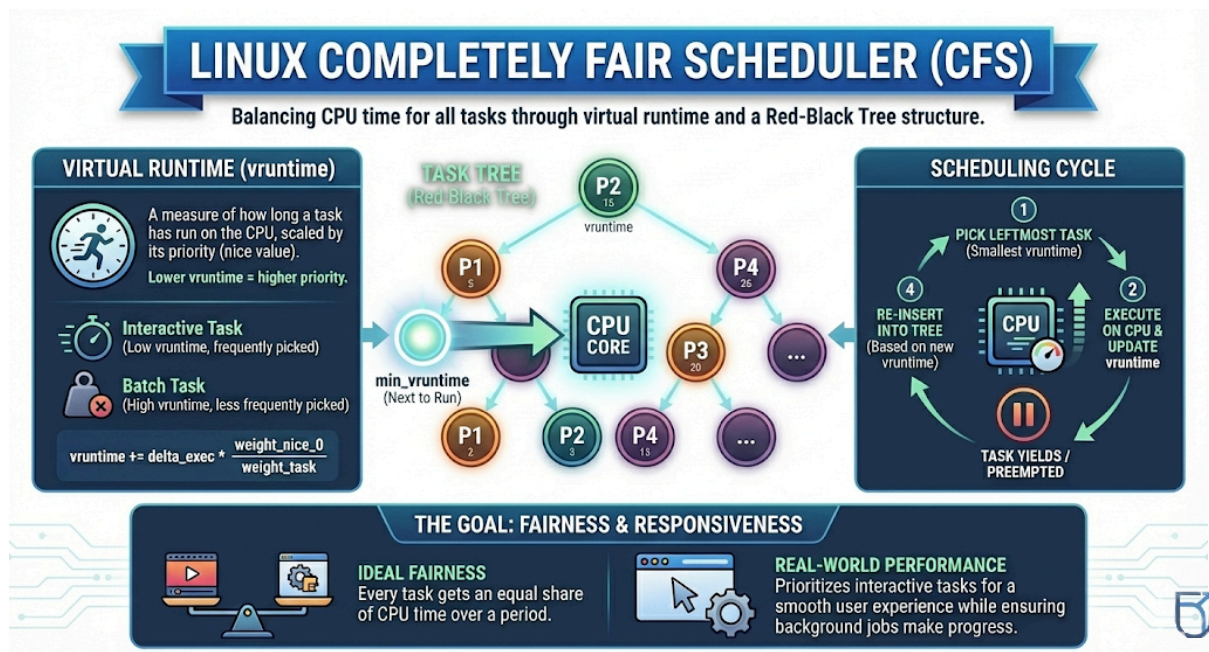
- **The Cheat:** A "smart" process runs for 99% of its time slice, then voluntarily yields just before the timer rings.
- **The Result:** The scheduler thinks it is an interactive job (because it didn't use the full quantum) and unfairly keeps it in the **High Priority Queue** forever.
- **Solution 1: Priority Boost (Aging):** Periodically move **ALL** jobs to the top queue. Everyone starts fresh.

- **Solution 2: Better Accounting:** Track the *total* CPU time used during a priority level. Once the *total* reaches the quantum limit (even if split across many small bursts), demote the process.

## MLFQ in Real Systems

- **Solaris / FreeBSD:** Use established MLFQ implementations with priority boosting.
- **Windows (NT/XP/Vista):** Uses a variation of MLFQ with dynamic priority boosts for foreground windows.
- **macOS:** Uses a Mach-based scheduler which is fundamentally a multi-level feedback queue.

## Linux Completely Fair Scheduler (CFS)



- **Philosophy:** Model an "ideal, precise multitasking CPU" where all N processes run simultaneously at 1/N speed.
- **Key Concept: vruntime (Virtual Runtime):**
  - A variable that tracks how long a process has *actually* run on the CPU.
  - **The Rule:** The process with the **lowest vruntime** (the one that has been treated most unfairly) is picked next.
- **Structure:** Uses a **Red-Black Tree** (not a list). The task with the lowest vruntime is always the leftmost node (fastest to find).

## Nice Values in Linux

In Linux, user priorities are mapped to "Nice" values. They control how much weight a process carries in the CFS vruntime calculation.

- **The Scale:** Ranges from **-20** (Highest Priority) to **+19** (Lowest Priority).
- **The Logic:** A "Niceness" of -20 means the process is *not nice* to others; it aggressively demands CPU. A +19 is *very nice*, letting everyone else go first.

- **Effect on CFS:**
  - Lower Nice value = Slower **vruntime** growth (The OS thinks it hasn't run enough, so it picks it more often).
  - Higher Nice value = Faster **vruntime** growth (The OS thinks it has run plenty).

## Windows Priority Scheduling

Windows uses a priority-based, preemptive scheduling system with 32 distinct levels.

- **Structure:**
  - **Levels 16–31 (Real-Time):** Fixed priorities. These tasks (like kernel drivers) always run before variable tasks. They do *not* change.
  - **Levels 1–15 (Variable):** Dynamic priorities. Used for standard user applications.
- **Dynamic Adjustments (The "Boosts"):**
  - **Foreground Boost:** The window currently in focus gets a longer time slice (Quantum x3) to ensure smooth typing/scrolling.
  - **I/O Boost:** After waiting for disk/network, a process gets a temporary priority bump so it can process the data immediately.
  - **Decay:** After the boost, the priority slowly drops back to its base level to prevent unfairness.

## Multi-Processor Scheduling Challenges

When you add more CPU cores, scheduling becomes exponentially harder.

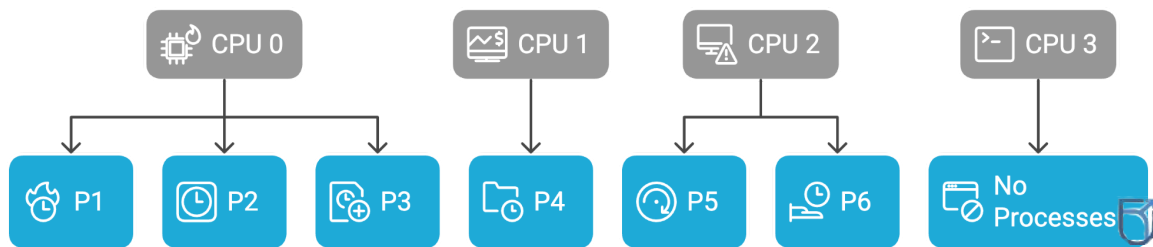
- **Load Balancing:** Ensuring no single CPU is overwhelmed while others sit idle.
- **Processor Affinity (Cache Locality):**
  - *Goal:* Keep a process running on the **same CPU core** as long as possible.
  - *Reason:* If a process moves to a new core, it loses the data stored in the previous core's fast L1/L2 Cache (Cache Miss), slowing it down.
- **Synchronization:** Multiple CPUs accessing the same Ready Queue requires "locking," which can become a bottleneck.

## Load Balancing Techniques

To solve the idle core problem, the OS moves processes between CPUs using two main strategies:

- **Push Migration:**
  - *Actor:* A specific system task checks load periodically.
  - *Action:* If it finds an imbalance, it **pushes** tasks from a busy CPU to an idle one.





- **Pull Migration:**
  - *Actor:* The Idle CPU itself.
  - *Action:* When a CPU has no work, it looks at other busy CPUs and **pulls** (steals) a waiting task to run.
- *Note:* Most systems (like Linux) use **both** techniques in parallel.

## CPU Affinity

Concept:

CPU Affinity (or processor affinity) is a scheduling property that dictates a process's preference to run on a specific CPU core or a set of cores over time, rather than switching frequently between different cores.

- **Why it matters:** When a process runs on a CPU, it populates that CPU's high-speed cache with its data and instructions. If the process moves to a different CPU, it must reload this data, causing performance delays.



**Types of Affinity:**

1. **Soft Affinity:**
  - The operating system *attempts* to keep a process on the same CPU, but it does not guarantee it.
  - If the system is under heavy load or rebalancing is needed, the process may be moved to another processor.

- *Common in:* Standard Linux scheduling (CFS).
2. **Hard Affinity:**
- The system strictly enforces that a process runs *only* on a specified subset of CPUs.
  - This is often set manually by system administrators (e.g., using the **taskset** command in Linux) to reserve cores for critical high-performance tasks.

**Benefits:**

- **Warm Cache:** Keeps the cache "warm" (filled with relevant data), reducing the need to fetch data from the slower main RAM.
- **Fewer Cache Misses:** significantly improves execution speed for memory-intensive applications.

## **NUMA Awareness (Non-Uniform Memory Access)**

Context:

In modern multiprocessor systems (especially servers), memory is not accessed uniformly. The system is divided into "nodes," where each CPU has its own local memory.

**Key Principles:**

- **Variable Access Time:** It is faster for a CPU to access its *local* memory than to access *remote* memory (memory attached to another CPU).
- **Scheduler Responsibility:** A NUMA-aware scheduler tries to allocate memory for a process and run that process on the same node.
- **Benefit:** Minimizes memory latency and bus traffic, leading to significant performance improvements in high-performance computing (HPC) and large database servers.

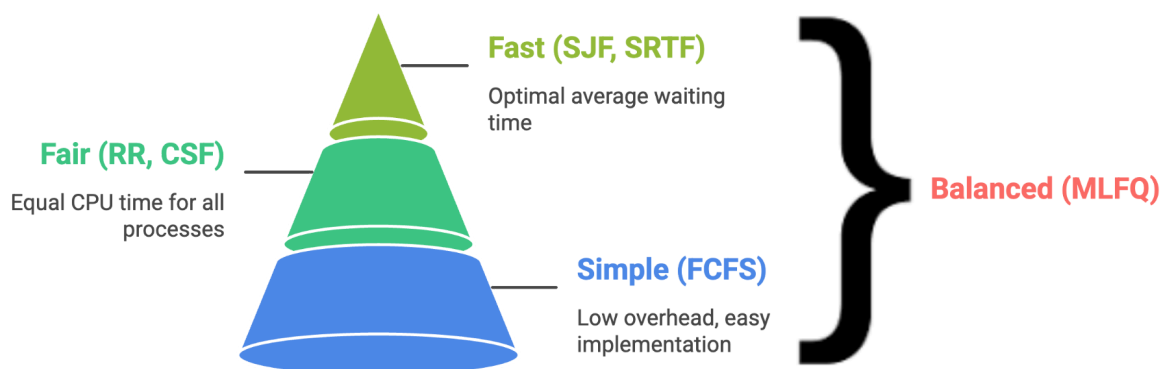
## **Scheduler Selection by System Type**

Different environments have different goals (latency vs. throughput), requiring different scheduling algorithms.

| System Type | Preferred Algorithm | Reason                                                                                                          |
|-------------|---------------------|-----------------------------------------------------------------------------------------------------------------|
| Web Servers | MLFQ / CFS          | Handles thousands of short, bursty requests (high concurrency) while preventing starvation of background tasks. |

|                            |                          |                                                                                                                                                        |
|----------------------------|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Scientific Clusters</b> | <b>SJF / FCFS</b>        | Focuses on <b>throughput</b> . Long-running calculation jobs benefit from simple non-preemptive scheduling to minimize context switching overhead.     |
| <b>Desktop OS</b>          | <b>MLFQ / CFS</b>        | Prioritizes <b>response time</b> (interactivity) so the mouse and UI feel smooth, while still running background tasks fairly.                         |
| <b>Real-Time Systems</b>   | <b>Priority (Strict)</b> | Must meet strict deadlines (e.g., airbag deployment). Uses preemptive priority scheduling with guarantees (Earliest Deadline First or Rate Monotonic). |

## The Scheduling Trade-Off Triangle



No single scheduling algorithm is perfect. System designers must balance three competing forces:

1. **Fairness:** Ensuring every process gets a chance to run (preventing starvation).
  - *Conflict:* strict fairness can hurt performance (e.g., Round Robin has high context switching).
2. **Performance (Throughput/Latency):** Maximizing the work done or minimizing wait times.
  - *Conflict:* Optimizing for throughput (SJF) often leads to unfairness (starvation of long jobs).
3. **Overhead:** The CPU time wasted by the scheduler itself (deciding what to run next).
  - *Conflict:* Complex algorithms (like  $O(\log N)$  CFS) afford better decisions but consume more CPU cycles than simple  $O(1)$  algorithms.

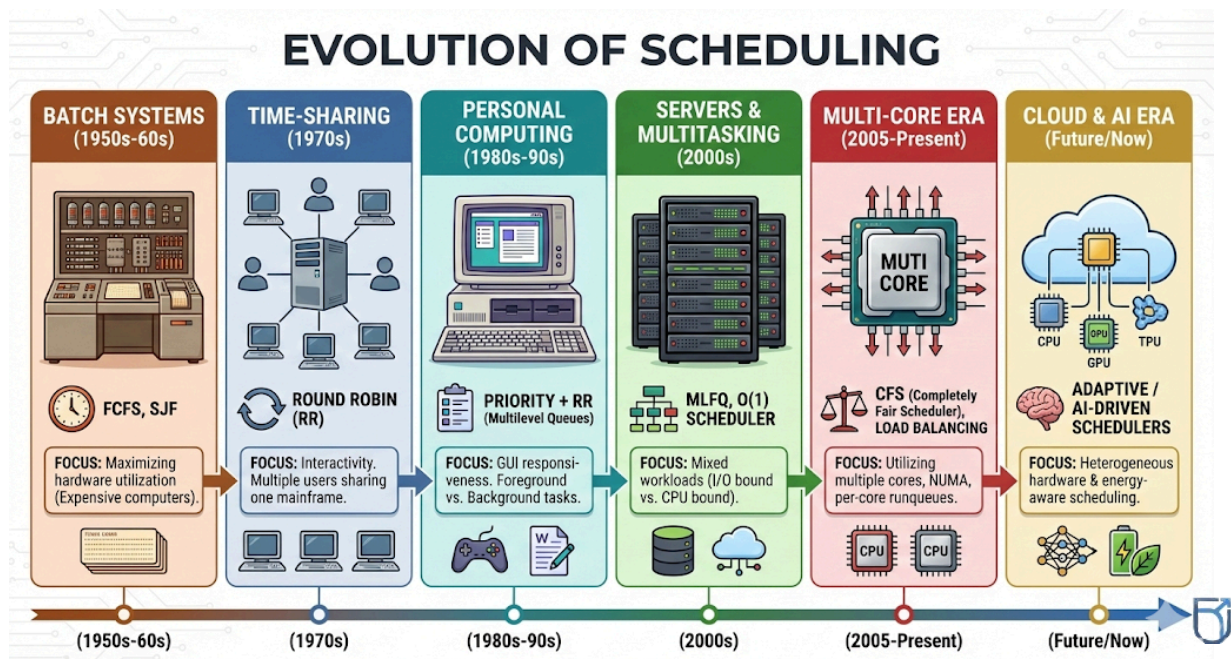
**Key Takeaway:** A general-purpose OS (like Windows or Linux) tries to sit in the middle of this triangle, compromising slightly on all three to provide a "good enough" experience for everyone.

## Evolution of Scheduling

A brief timeline of how scheduling logic has adapted to hardware changes:

### 1. Batch Systems (1950s-60s):

- **Algo:** **FCFS (First-Come, First-Served), SJF.**
- **Focus:** Maximizing hardware utilization (computers were expensive; humans were cheap).



### 2. Time-Sharing (1970s):

- **Algo:** **Round Robin (RR).**
- **Focus:** Interactivity. Allowing multiple users to use one mainframe simultaneously.

### 3. Personal Computing (1980s-90s):

- **Algo:** **Priority + RR (Multilevel Queues).**
- **Focus:** GUI responsiveness. Foreground tasks (Word, Games) got higher priority than background tasks.

### 4. Servers & Multitasking (2000s):

- **Algo:** **MLFQ (Multi-Level Feedback Queue), O(1) Scheduler.**
- **Focus:** Handling mixed workloads (I/O bound vs. CPU bound) efficiently.

### 5. Multi-Core Era (2005-Present):

- **Algo:** **CFS (Completely Fair Scheduler), Load Balancing.**
- **Focus:** utilizing multiple cores efficiently, handling NUMA, and per-core runqueues.

### 6. Cloud & AI Era (Future/Now):

- *Algo*: **Adaptive / AI-driven Schedulers**.
- *Focus*: Scheduling for heterogeneous hardware (CPU + GPU + TPU) and optimizing for power consumption (energy-aware scheduling).

## NUMERICAL QUESTIONS

### Problem 1: Algorithm Comparison

#### Scenario

Three processes with the following arrival and burst times:

- **P1**: Arrival Time = 0, Burst Time = 24
- **P2**: Arrival Time = 1, Burst Time = 3
- **P3**: Arrival Time = 2, Burst Time = 3

#### Task

Calculate the **Average Waiting Time** for:

1. FCFS (First-Come, First-Served)
2. SJF (Shortest Job First – Non-Preemptive)
3. Round Robin (Time Quantum = 5)

### 1. FCFS (First-Come, First-Served)

#### Core Logic

- **Principle**: The "Queue" Logic. Processes are executed strictly in arrival order.
- **Non-Preemptive**: Once P1 starts, it holds the CPU until it finishes.
- **The Flaw (Convoy Effect)**: P1 is a massive process (24ms). P2 and P3 are tiny (3ms), but they are forced to wait behind P1.

#### Execution Sequence

- **Time 0**: **P1** arrives and starts immediately. (Runs 0–24).
- **Time 24**: P1 finishes. **P2** is next. (Runs 24–27).
- **Time 27**: P2 finishes. **P3** is next. (Runs 27–30).

#### Gantt Chart



| Time Interval | Process ID |
|---------------|------------|
| 0 – 24        | P1         |
| 24 – 27       | P2         |
| 27 – 30       | P3         |

### Calculation Table

- **Formula:**  
Waiting Time (WT) = Start Time - Arrival Time

| Process | Arrival Time (AT) | Burst Time (BT) | Start Time | Completion Time | Waiting Time (WT) |
|---------|-------------------|-----------------|------------|-----------------|-------------------|
| P1      | 0                 | 24              | 0          | 24              | $0 - 0 = 0$       |
| P2      | 1                 | 3               | 24         | 27              | $24 - 1 = 23$     |
| P3      | 2                 | 3               | 27         | 30              | $27 - 2 = 25$     |

### Final Average

- **Average Waiting Time:**  $(0 + 23 + 25) / 3 = 16.0 \text{ ms}$

## 2. SJF (Shortest Job First – Non-Preemptive)

### Core Logic

- **Principle:** Select the process with the shortest burst time.
- **The Trap:** This is **Non-Preemptive**. At Time 0, only P1 is available. It *must* run.
- **Result:** By the time P1 finishes (Time 24), P2 and P3 have been waiting for a long time. SJF fails to help here because it couldn't stop P1.

## Execution Sequence

- **Time 0: P1** starts (only process present). Runs 0–24.
- **Time 24:** P1 finishes. Available: P2 (3ms) and P3 (3ms).
  - *Tie-Breaker:* P2 arrived at T=1, P3 at T=2. **P2 wins.**
- **Time 24–27: P2** runs.
- **Time 27–30: P3** runs.

## Gantt Chart

| Time Interval | Process ID | Note                        |
|---------------|------------|-----------------------------|
| 0 – 24        | P1         | Locked CPU (Non-preemptive) |
| 24 – 27       | P2         | Tie-breaker winner          |
| 27 – 30       | P3         | Last remaining              |

## Calculation table-

| Process | AT | BT | Completion Time | Turnaround Time (CT-AT) | Waiting Time (TAT-BT) |
|---------|----|----|-----------------|-------------------------|-----------------------|
| P1      | 0  | 24 | 24              | 24                      | 24 - 24 = 0           |

|           |   |   |    |    |               |
|-----------|---|---|----|----|---------------|
| <b>P2</b> | 1 | 3 | 27 | 26 | $26 - 3 = 23$ |
| <b>P3</b> | 2 | 3 | 30 | 28 | $28 - 3 = 25$ |

### Final Average

- **Average Waiting Time:**  $(0 + 23 + 25) / 3 = 16.0$ ;

### 3. Round Robin (Time Quantum = 5)

#### Core Logic

- **Principle:** The "Fairness" Logic. Processes are sliced into 5ms chunks.
- **Goal:** Allow the short processes (P2, P3) to finish *before* the long process (P1) is done.

#### Execution Sequence

- **0–5:** **P1** runs (5ms). Rem: 19. Queue: **P2 → P3 → P1**.
- **5–8:** **P2** runs. Only needs 3ms. **Finishes**. Queue: **P3 → P1**.
- **8–11:** **P3** runs. Only needs 3ms. **Finishes**. Queue: **P1**.
- **11–30:** **P1** runs remaining 19ms. (Since no one else is waiting, it runs straight through).

#### Gantt Chart

| Time Interval | Process ID | Reason for Switch |
|---------------|------------|-------------------|
| <b>0 – 5</b>  | <b>P1</b>  | Quantum Expired   |
| <b>5 – 8</b>  | <b>P2</b>  | Finished Early    |

|         |    |                    |
|---------|----|--------------------|
| 8 – 11  | P3 | Finished Early     |
| 11 – 30 | P1 | Runs to Completion |

### Calculation Table

- **Formula:**

$$WT = TAT - BT$$

| Process | AT | BT | Completion Time (CT) | Turnaround Time (CT-AT) | Waiting Time (TAT-BT)  |
|---------|----|----|----------------------|-------------------------|------------------------|
| P1      | 0  | 24 | 30                   | $30 - 0 = 30$           | $30 - 24 = \mathbf{6}$ |
| P2      | 1  | 3  | 8                    | $8 - 1 = 7$             | $7 - 3 = \mathbf{4}$   |
| P3      | 2  | 3  | 11                   | $11 - 2 = 9$            | $9 - 3 = \mathbf{6}$   |

- 

### Final Average

- **Average Waiting Time:**  $(6 + 4 + 6) / 3 = 5.333 \text{ ms}$

### Comparative Analysis & Conclusion:

| Algorithm | Avg Waiting Time | Analysis                                                         |
|-----------|------------------|------------------------------------------------------------------|
| FCFS      | 16.0 ms          | <b>Worst.</b> P2 and P3 suffered massive delays (Convoy Effect). |

|                    |                |                                                                                              |
|--------------------|----------------|----------------------------------------------------------------------------------------------|
| <b>SJF</b>         | 16.0 ms        | <b>Failed.</b> Identical to FCFS because the long job arrived first and couldn't be stopped. |
| <b>Round Robin</b> | <b>5.33 ms</b> | <b>Best.</b> The time slice allowed P2 and P3 to "cut in line" and finish very quickly.      |

**Final Verdict: Round Robin** is the best choice for this workload. It significantly reduces the waiting time by allowing shorter processes to execute earlier, making it ideal for interactive systems.